

Instituto Federal
Campus Goiânia

Bacharelado em Sistemas de Informação

POO II

Prof. Dory Gonzaga Rodrigues



Ementa

Introdução

INTRODUÇÃO - JDBC

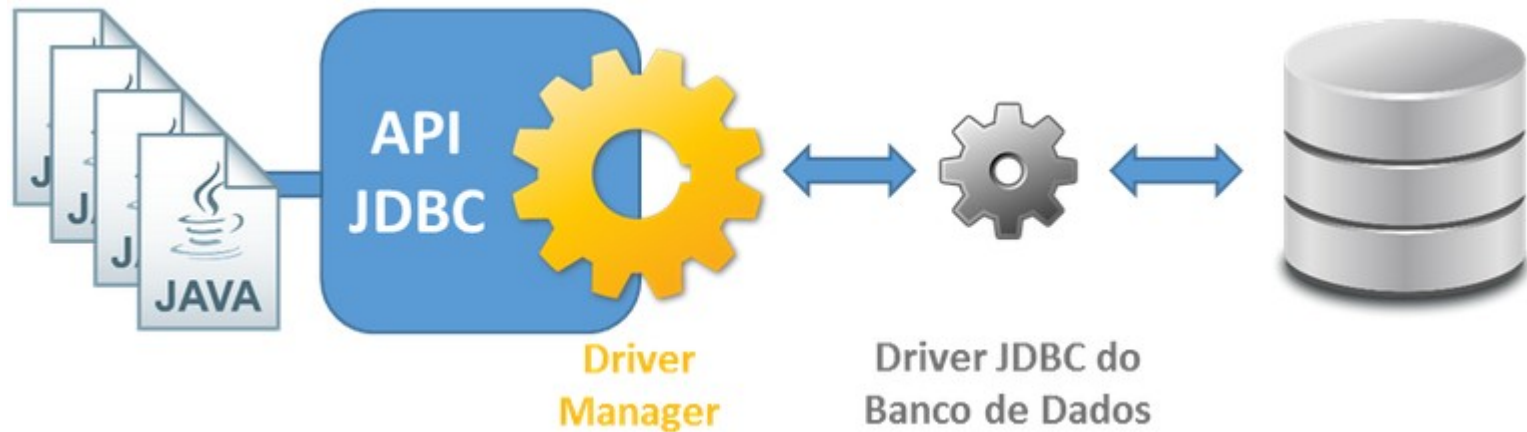


JDBC

Introdução

O recurso base do Java para a persistência de dados é a API JDBC.

De forma simples, podemos definir essa API como o meio encontrado pelo Java para que o seu programa possa conversar com o banco de dados.



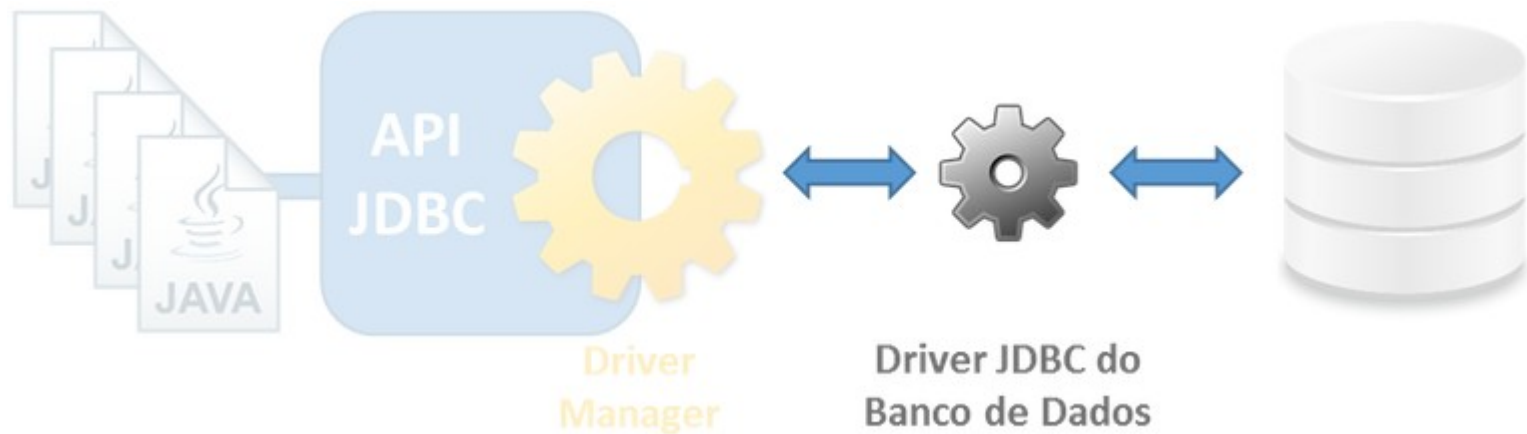
É com essa API que conseguimos nos conectar ao banco e enviar ou recuperar informações.

Basicamente, precisamos conhecer as principais classes e métodos da API e ter o driver JDBC fornecido pela empresa que criou o SGBD (por exemplo, o **driver JDBC do Postgres**).

JDBC

O que é um Driver ?

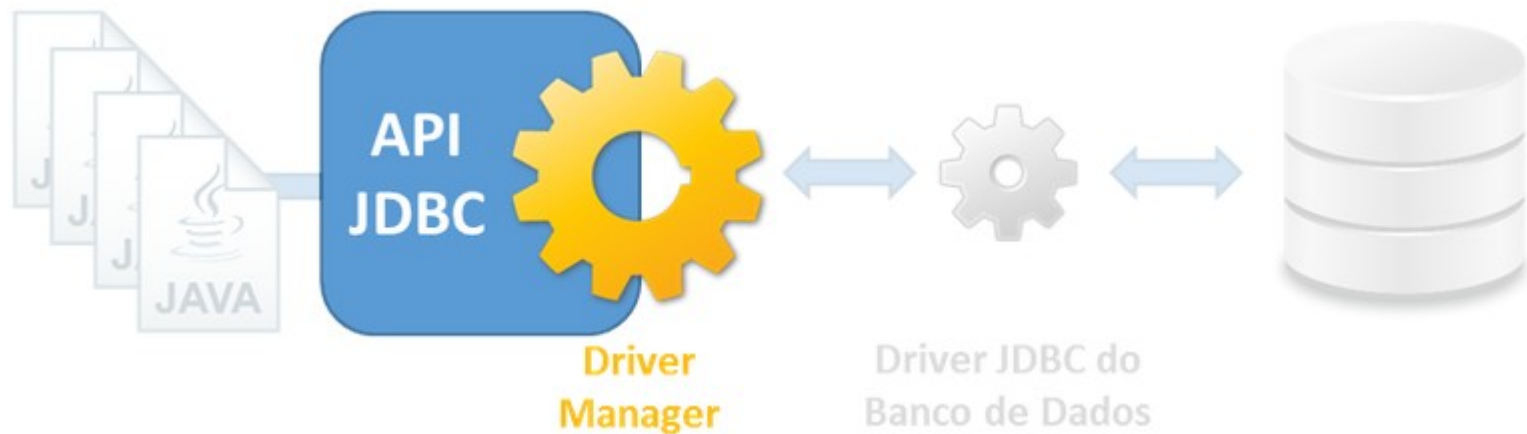
Além de atuar como uma **interface** entre os SGBDs e as aplicações, também pode ser considerado como um tradutor que ajuda na definição das mensagens binárias trocadas com um protocolo de um SGBD.



JDBC

API JDBC

Para desenvolver uma aplicação baseada em uma especificação JDBC é preciso entender algumas das principais classes e interfaces desta API.



JDBC

Pacote java.sql

Esse pacote oferece a biblioteca Java de acesso e processamento de dados em uma fonte de dado. As classes e interfaces mais importantes são:

Categoria	Classe ou Interface	Fornecido por	Comentário
Mapeamento entre tipos de dados SQL e tipos de dados Java	Classe Date Classe Time Classe TimeStamp	java.sql	Valor SQL para Data Valor SQL para Time Valor SQL para TimeStamp
Fornecimento de metadados sobre o banco de dados	Interface DatabaseMetadata Interface ResultSetMetadata	Fabricante do SGBD	Informação sobre o BD
Geração de Exceções	Classe SQLException Classe SQLWarning	java.sql	Erro de acesso ao BD Alertas de acesso ao BD

JDBC

Pacote java.sql

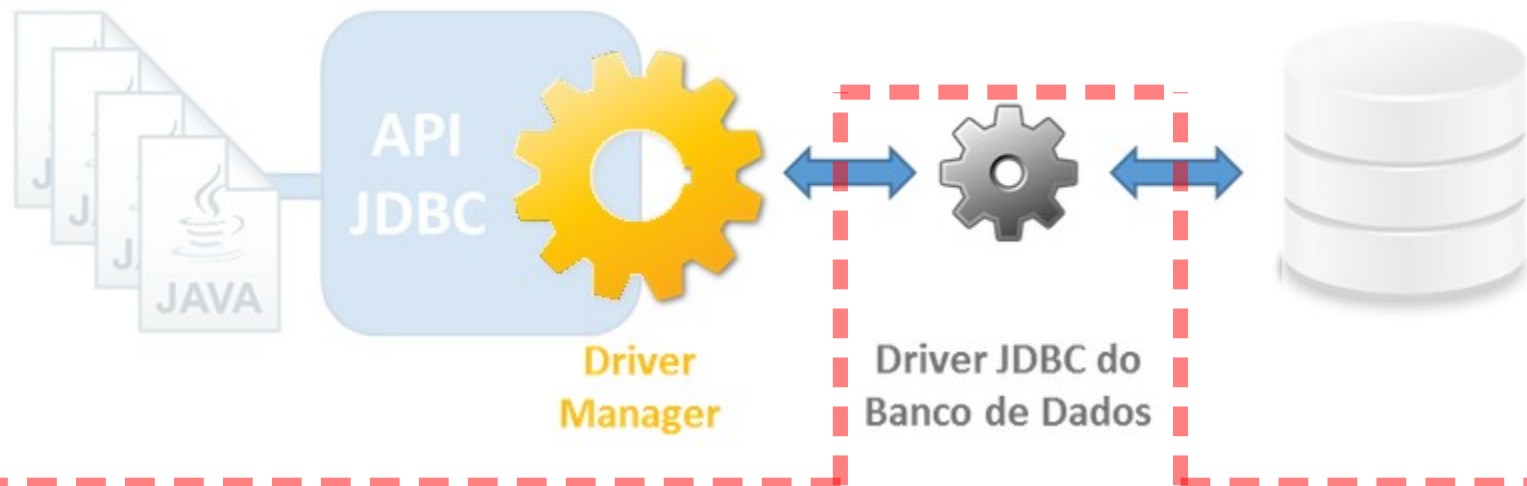
Esse pacote oferece a biblioteca Java de acesso e processamento de dados em uma fonte de dado. As classes e interfaces mais importantes são:

Categoria	Componente	Funcionalidade
Carrega um driver	java.sql.DriverManager	Gerencia um ou vários drivers Cria uma conexão com o banco
Representa um driver de conexão	java.sql.Driver	Fornecer uma conexão e informações do driver. Cada driver é representado por uma URL
Representa a conexão	java.sql.Connection	Representa a conexão com o banco e a sessão de conexão
Declaração do comando SQL	java.sql.Statement	Utilizado para montar o comando SQL para ser executado e
Representa o conjunto de dados retornado pelo Banco	java.sql.ResultSet	Utilizado para manipular os dados retornados na execução de uma consulta

JDBC

A classe DriverManager

Cada driver que se deseja usar precisa ser registrado nessa classe, pois oferece métodos estáticos para gerenciar um driver JDBC.



Para a aplicação reconhecer a comunicação com o banco de dados escolhido, é preciso obter a biblioteca (extensão .jar) e importá-la (adicionando ao classpath do projeto).

**JDBC**

Principal método da classe DriverManager

getConnection(String url, String user, String password)

Tenta estabelecer uma conexão com o URL do banco de dados fornecido. O DriverManager tenta selecionar um driver apropriado do conjunto de drivers JDBC registrados, em caso afirmativo, retorna a conexão.

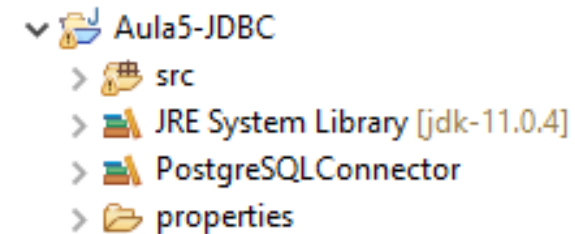
Para uma portabilidade máxima, um aplicativo deve especificar apenas uma propriedade por vez. Sendo assim, passamos separadamente os seguintes parâmetros:

- | | |
|---------|-----------------------------|
| url | - url do banco de dados |
| usuário | - usuário do banco de dados |
| senha | - a senha do usuário |

JDBC

Exemplo – Conexão via JDBC

```
1 package db;
2+ import java.io.IOException;
8
9 public class DB {
10
11     private static Connection conexao;
12
13- public static Properties getPropriedades() throws IOException {
14         Properties propriedades = new Properties();
15         FileInputStream file = new FileInputStream("../properties/db.properties");
16         propriedades.load(file);
17         return propriedades;
18     }
19
20- public static Connection getConexao() throws SQLException, IOException{
21         String host;
22         String user;
23         String password;
24         Properties propriedades = DB.getPropriedades();
25
26         host = propriedades.getProperty("host");
27         user = propriedades.getProperty("user");
28         password = propriedades.getProperty("password");
29         conexao = DriverManager.getConnection(host, user, password);
30
31         return conexao;
32     }
33
34- public static void CloseConnection () throws SQLException{
35
36         if (conexao != null) {
37             conexao.close();
38         }
39     }
40 }
```



**Não se esqueça:
importe o driver
no projeto**



JDBC

A classe Connection

Esta classe representa uma conexão (sessão) com um banco de dados específico. As instruções SQL são executadas e os resultados são retornados no contexto de uma conexão.

'O banco de dados de um objeto de conexão é capaz de fornecer informações que descrevem suas tabelas, sua gramática de SQL suportada, seus procedimentos armazenados, as capacidades desta conexão e assim por diante.

A seguir veremos os principais métodos da classe !

**JDBC**

Principais métodos da classe Connection

Método setAutoCommit()

Esse método aceita como parâmetro um valor booleano (true ou false), no qual consegue definir se todas as instruções executadas serão gravadas (comitadas).

Por padrão, quando finalizada uma instrução de INSERT, UPDATE ou DELETE a operação de gravação (commit) é executada automaticamente sem a necessidade de invocar o método setAutoCommit.

Agora, para instruções de SELECT, a gravação dos dados ocorre quando o conjunto de dados associados é fechado, ou seja, quando invocado o método close().

Método commit()

Efetiva todas as alterações feitas desde o último commit/rollback e libera todos os bloqueios de banco de dados atualmente mantidos por este objeto de Conexão.

Este método deve ser usado somente quando o modo de confirmação automática foi desabilitado.

**JDBC**

Principais métodos da classe Connection

Método rollback()

Usado para retornar uma transação, sendo muito usado dentro de blocos catch em caso de alguma operação gerar erro.

Método close()

Geralmente é inserido dentro do bloco finally para realizar sempre a sua execução, pois esse método serve para fechar e liberar imediatamente um objeto Connection.

Método isClosed()

Serve para verificar se o objeto Connection está fechado.

**JDBC**

Principais métodos da classe Connection

Método `prepareStatement(String sql)`

- Cria um objeto `PreparedStatement` para enviar instruções SQL parametrizadas para o banco de dados.
- Uma instrução SQL com ou sem parâmetros de entrada (IN) pode ser pré-compilada e armazenada em um objeto `PreparedStatement`.

Método `prepareCall(String sql)`

- Cria um objeto `CallableStatement` para chamar procedimentos armazenados no banco de dados.
- O objeto `CallableStatement` fornece métodos para configurar seus parâmetros IN e OUT e métodos para executar a chamada para um procedimento armazenado.

**JDBC**

Classe PreparedStatement

Um objeto PreparedStatement representa uma declaração SQL pré-compilada.

Este objeto pode então ser usado para executar esta declaração com eficiência várias vezes.



JDBC

Principais métodos da classe PreparedStatement

Método `setInt`,
`setFloat`,
`setDate`,
`setString`,
`setTime` e assim por diante

Utilizado para definir os valores dos parâmetros de entrada da SQL (IN). Deve especificar o tipo compatível com o tipo SQL definido no parâmetro de entrada.

Por exemplo:

```
PreparedStatement st = con.prepareStatement(  
    "INSERT INTO aluno (nome, idade) "  
    + "VALUES (?, ?)"  
);
```

```
st.setString(1, "João");  
st.setInt(2, 25);
```

Onde:

parâmetro do tipo **SQL VARCHAR**, o método `setString` deve ser usado.
parâmetro do tipo **SQL INTEGER**, o método `setInt` deve ser usado.

**JDBC**

Principais métodos da classe PreparedStatement

Método execute()

Executa a instrução SQL neste objeto PreparedStatement, que **pode ser qualquer tipo de declaração SQL**.

Método executeQuery()

Executa instrução SQL – DML (**SELECT**) neste objeto PreparedStatement.

Retorna o objeto **ResultSet** gerado pela consulta.

Método executeUpdate()

Executa instrução SQL – DML (**INSERT, UPDATE ou DELETE**) ou instrução DDL que não retorne nada.

JDBC

Exemplo – PreparedStatement

```
1 package application;
2
3 import java.io.IOException;
17
18 public class Program {
19
20     public static void main(String args[]) throws SQLException, IOException, ParseException {
21
22         Connection con = DB.getConexao();
23         System.out.println("*** Conexão Realizada ***");
24
25         PreparedStatement st = con.prepareStatement(
26             "INSERT INTO aluno (nome, sexo, dt_nasc) "
27             + "VALUES (?, ?, ?)"
28         );
29
30         st.setString(1, "João");
31         st.setString(2, "Masculino");
32
33         Calendar c = new GregorianCalendar(1990, 2, 15);
34         st.setDate(3, new Date(c.getTimeInMillis()));
35
36         int linhas = st.executeUpdate();
37
38         System.out.println("Linhas inseridas: " + linhas);
39
40         st.close();
41         DB.CloseConnection();
42
43         System.out.println("*** Conexão Encerrada ***");
44     }
45 }
```

JDBC

Classe ResultSet

Representa o conjunto de resultados de uma tabela no banco de dados.

Esse objeto mantém o cursor apontando para a sua linha atual de dados, sendo que seu início fica posicionado na primeira linha. Um [objeto ResultSet](#) fornece os métodos getters referentes aos tipos de dados do banco: [getInt](#), [getString](#), [getDouble](#), [getFloat](#), [getLong](#) etc

Recuperamos os valores usando [número do índice](#) ou [nome da coluna](#):

```
String sql = "SELECT * FROM aluno";
ResultSet rs = st.executeQuery(sql);

while (rs.next())
{
    System.out.println(
        rs.getInt(1) + rs.getString(2) + rs.getString(3) + rs.getDate("dt_nasc")
    );
}
```

	id [PK] bigint	nome character varying	sexo character varying	dt_nasc date
1	1	Dory	Masculino	1976-03-31
2	2	Caroline	Feminino	2000-12-24

JDBC

Exemplo – ResultSet

```
1 package application;
2
3+ import java.io.IOException;
10
11 public class Program {
12
13-     public static void main(String args[]) throws SQLException, IOException {
14
15         Connection con = DB.getConexao();
16         System.out.println("*** Conexão Realizada ***");
17
18         Statement st = con.createStatement();
19
20         String sql = "SELECT * FROM aluno";
21         ResultSet rs = st.executeQuery(sql);
22
23         while (rs.next())
24         {
25             System.out.println(
26                 rs.getInt(1) + rs.getString(2) + rs.getString(3) + rs.getDate("dt_nasc")
27             );
28         }
29
30         rs.close();
31         st.close();
32         DB.CloseConnection();
33
34         System.out.println("*** Conexão Encerrada ***");
35     }
36 }
```

JDBC

Resumo JDBC – Ciclo de Vida